# Exercise 9

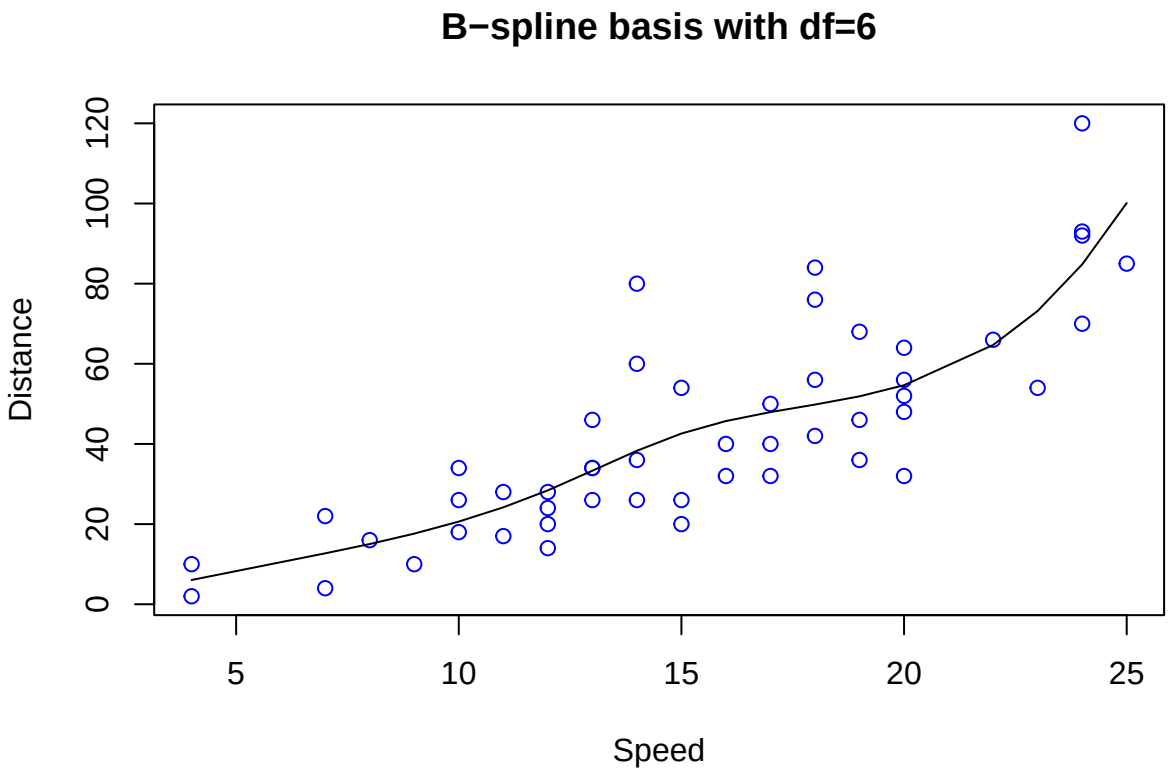
Advanced Methods for Regression and Classification

Kyzer Gerez

10.12.2025

(1) Construct a B-spline basis for `speed` by using the function `bs()` from the package `splines`, and fit a linear model. Select the degrees of freedom in a way to obtain a plausible fit.

```r
library(splines)
library(datasets)
data("cars")

# fit linear model using bs() then predict
df <- 6
model.bs <- lm(dist ~ bs(speed, df=df),
               data=cars)
pred.bs <- predict(model.bs, newdata=cars)

# plotting
plot(cars$speed, cars$dist, col="blue", type="p",
     main=paste("B-spline basis with df=", df, sep=""),
     xlab="Speed",
     ylab="Distance")
lines(cars$speed, pred.bs)
```



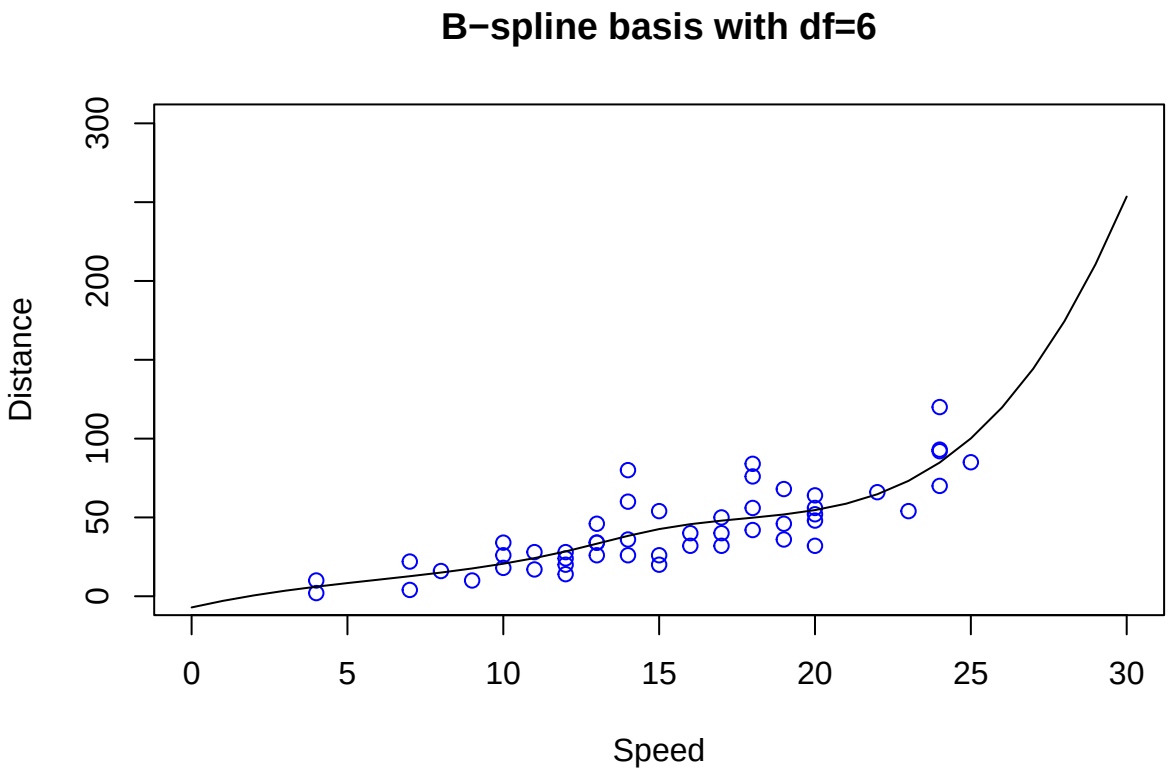

(2) Construct B-splines for a speed range [0, 30] by using the knots from (1), and use the linear model to obtain predictions for the new domain.

```
# predict on [0,30]
x.new <- data.frame(speed = seq(0, 30, length.out = 31))
pred.bs.ext <- predict(model.bs, newdata=x.new)
```

```
## Warning in bs(speed, degree = 3L, knots = c(12, 15, 19), Boundary.knots = c(4,
## : some 'x' values beyond boundary knots may cause ill-conditioned bases
```

(3) Plot the data `dist` versus `speed` by extending the plot coordinates appropriately, and show the prediction from (2) as a line. Does the prediction fulfill the requirements stated above?

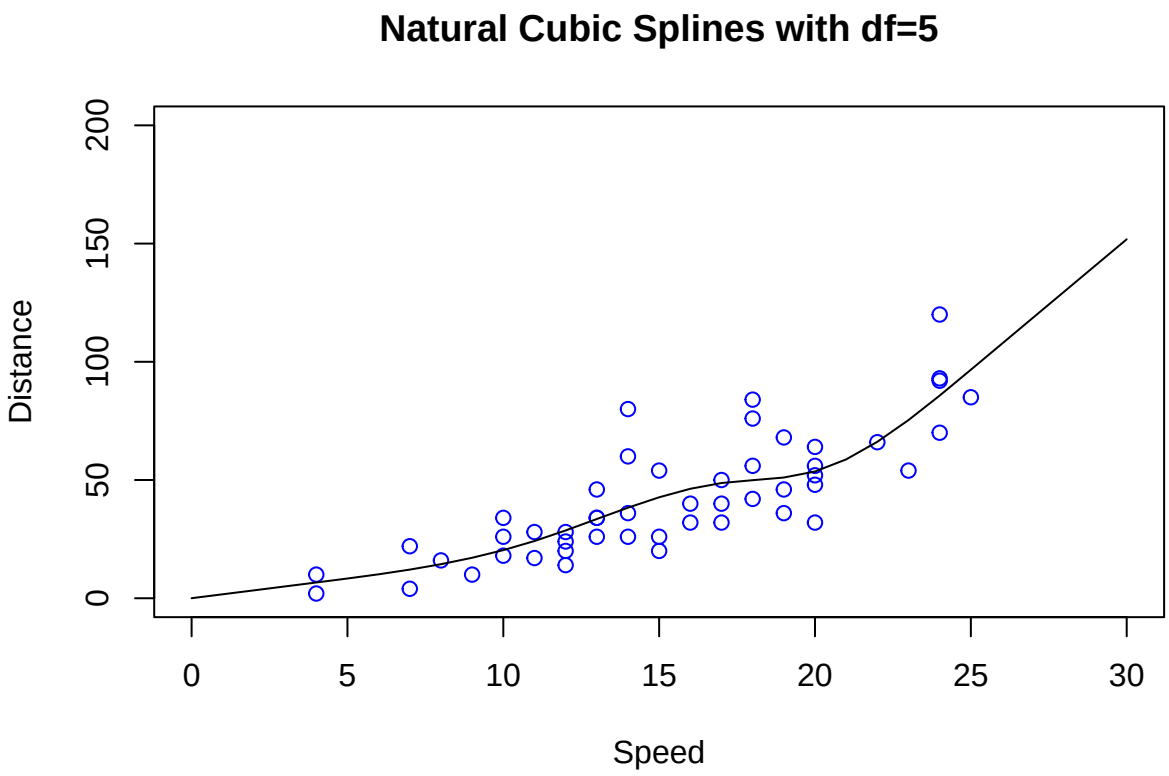
```
# plotting
plot(cars$speed, cars$dist, col="blue", type="p",
     main=paste("B-spline basis with df=", df, sep=""),
     xlab="Speed",
     ylab="Distance",
     xlim=c(0,30),
     ylim=c(0, 300))
lines(x.new$speed, pred.bs.ext)
```




The prediction does not fulfill the requirements. The model predicted negative values for speeds 0 and 1.

(4) Same as (1)-(3), but for `ns()`.

```
# fit linear model using bs() then predict
df.ns <- 5
model.ns <- lm(dist ~ ns(speed, df=df.ns),
               data=cars)
pred.ns <- predict(model.bs, newdata=cars)

# predict on [0,30]
pred.ns.ext <- predict(model.ns, newdata=x.new)

# plotting
plot(cars$speed, cars$dist, col="blue", type="p",
     main=paste("Natural Cubic Splines with df=", df.ns, sep=""),
     xlab="Speed",
     ylab="Distance",
     xlim=c(0,30),
     ylim=c(0, 200))
lines(x.new$speed, pred.ns.ext)
```




The predictions using the model created from `ns()` are better than those from `bs()` in that there are no negative predictions. However, the predicted distance at speed=0 is still non-zero, though it is a small value.

Additionally, the extrapolated values for the high end are less reasonable compared to the model created from `bs()`. There shouldn't be a linear relationship between stopping distance and speed.

(5) Use smoothing splines as implemented in `smooth.spline()`, and select the optimal tuning parameter by the internal cross-validation. What are the resulting degrees of freedom?

```
smooth.spline.cv <- smooth.spline(cars$speed, cars$dist, cv=TRUE)
```

```
## Warning in smooth.spline(cars$speed, cars$dist, cv = TRUE): cross-validation
## with non-unique 'x' values seems doubtful
```

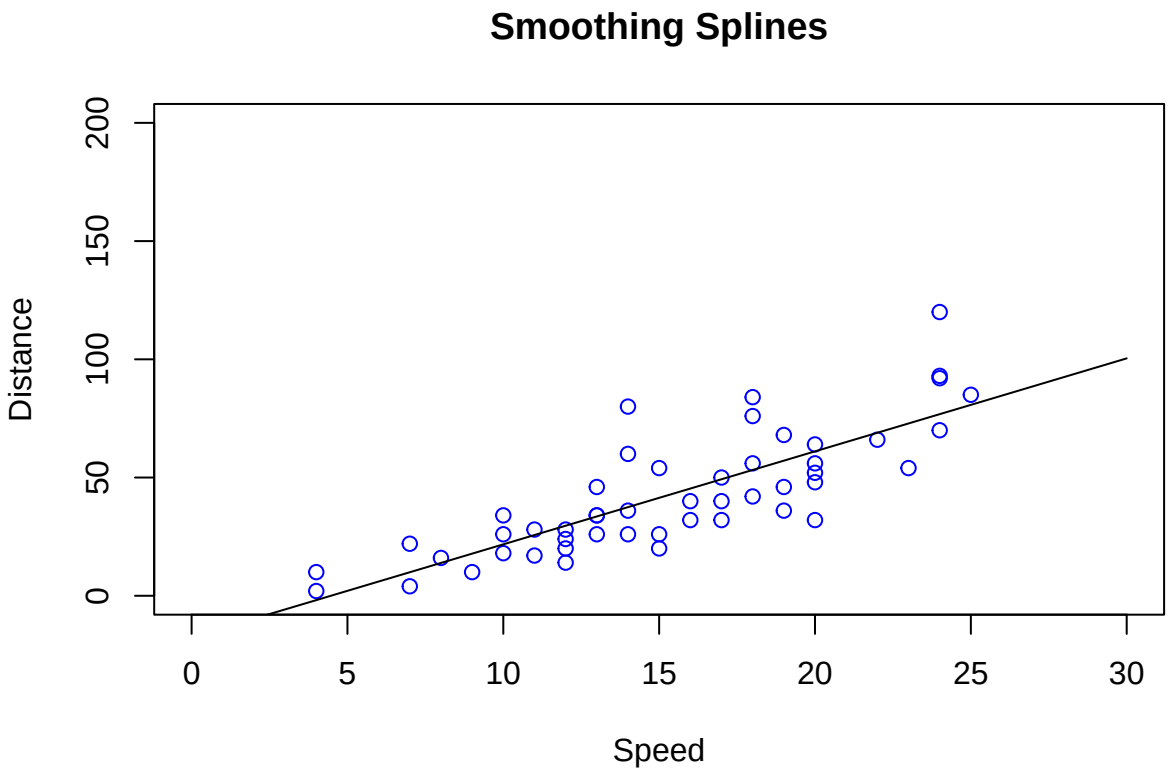
```
pred.smooth <- predict(smooth.spline.cv, newdata=cars)
```

The equivalent number of degrees of freedom is approximately 2.0000112. The smoothing parameter, `spar`, is 1.4689148.

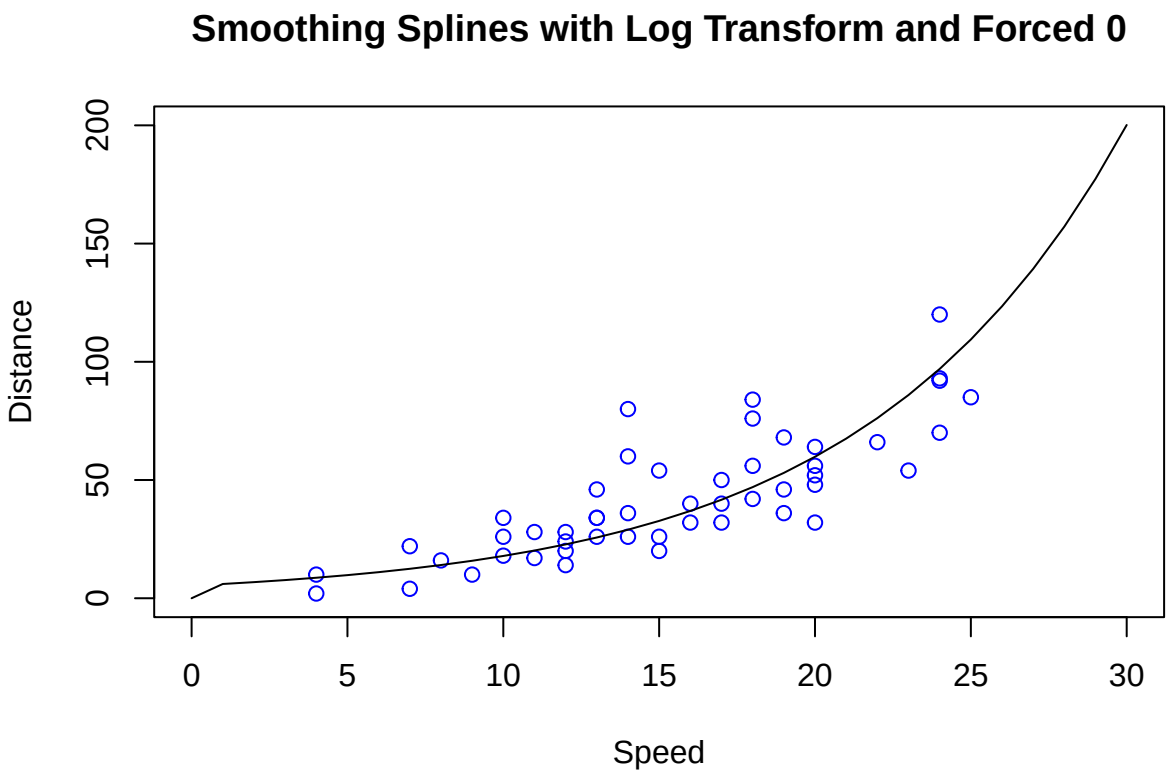
(6) Take the model from (5) to predict for the extended range [0, 30], and show the predictions in a plot. What can we do to fulfill the requirements stated above?

```
# predict on [0,30]
pred.smooth.ext <- predict(smooth.spline.cv, x=x.new)

# plotting
plot(cars$speed, cars$dist, col="blue", type="p",
     main="Smoothing Splines",
     xlab="Speed",
     ylab="Distance",
     xlim=c(0,30),
     ylim=c(0, 200))
```

```r
lines(pred.smooth.ext$x$speed, pred.smooth.ext$y$speed)
```




```r
# use log transform and force 0 dist at speed=0 to meet requirements
model.smooth <- smooth.spline(cars$speed, log(cars$dist),
                              spar=smooth.spline.cv$spar,
                              df=smooth.spline.cv$df)
pred.smooth.force <- predict(model.smooth, x=x.new)
pred.smooth.force$y$speed <- exp(pred.smooth.force$y$speed)
pred.smooth.force$y$speed[1] <- 0

# plotting
plot(cars$speed, cars$dist, col="blue", type="p",
     main="Smoothing Splines with Log Transform and Forced 0",
     xlab="Speed",
     ylab="Distance",
     xlim=c(0,30),
     ylim=c(0, 200))
lines(pred.smooth.force$x$speed, pred.smooth.force$y$speed)
```




Positive predictions can be enforced by using a log transform. A simple solution to predicting a stopping distance of 0 at speed=0 is to manually set the prediction to 0 at that speed.

The extrapolation is now reasonable at the high end, but there's a sudden drop in stopping distance from 6.0308098 at speed=1 to 0 at speed=0.